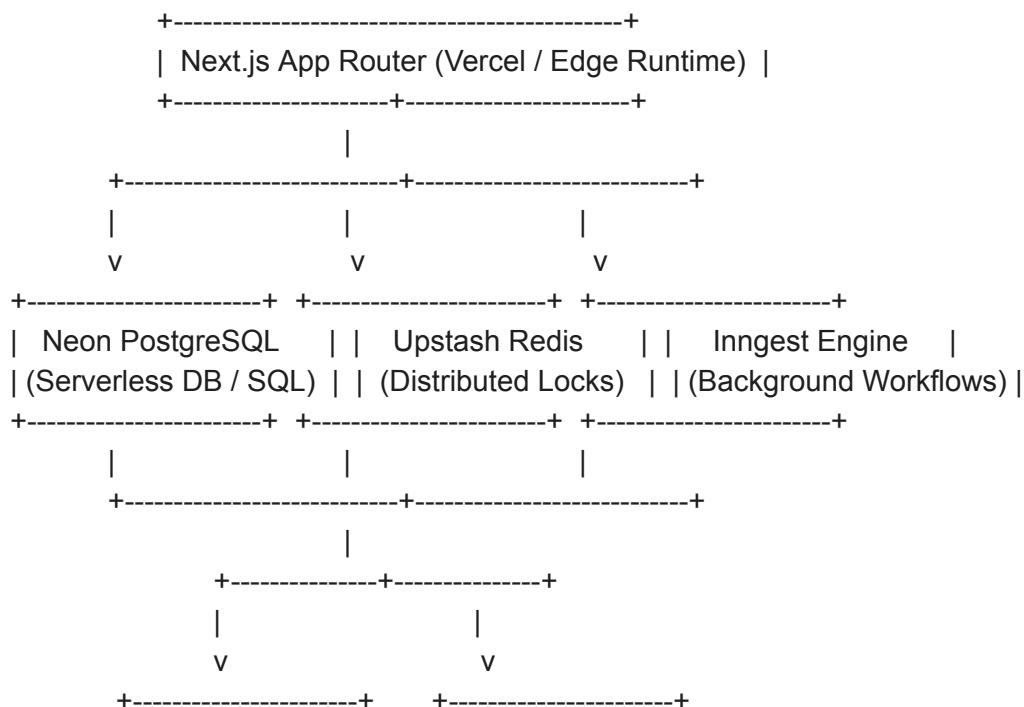
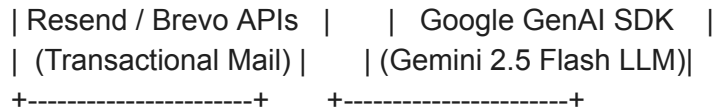


Enterprise Healthcare Appointment and Follow-Up Platform: Systems Architecture, Concurrency Engineering, and Multilingual Accessibility Specification

Infrastructure Architecture & Zero-Cost Technology Topology

Constructing an enterprise-grade healthcare appointment and follow-up platform operating under zero-cost infrastructure parameters requires a serverless architecture designed to eliminate recurring operational expenses while maintaining operational resilience. The system architecture decouples state storage, distributed caching, event orchestration, transactional communications, and artificial intelligence inference into dedicated cloud tiers that offer perpetual free operational envelopes.





The database foundation utilizes Neon PostgreSQL, a fully managed serverless relational database that separates storage from compute infrastructure¹. Neon’s free tier provides 0.5 GB of persistent data storage per project and 100 Compute Unit hours (CU-hrs) monthly, automatically scaling active compute resources to zero after five minutes of inactivity¹. While cold starts on autosuspended compute can introduce a query latency penalty ranging from 500 milliseconds to 2 seconds⁴, the application layer neutralizes this by executing queries through the stateless @neondatabase/serverless driver over HTTP and WebSockets. This approach bypasses TCP connection setup overhead during transient serverless invocations and preserves raw performance.

Real-time concurrency controls, session management, and transient state caching rely on Upstash Redis accessed via HTTP REST protocols⁵. Upstash provides a perpetual free tier supplying 256 MB of memory and 500,000 commands per month⁵. Because traditional Redis TCP connections are poorly suited for stateless serverless environments, the Upstash REST SDK allows edge handlers to perform atomic distributed locking without risking connection pool exhaustion⁵.

Asynchronous event processing, background retry sagas, and long-running workflows—such as doctor leave propagation—are orchestrated using Inngest⁷. Inngest functions on an event-driven architecture that triggers HTTP endpoints in response to application events, eliminating the need for continuously running queue consumers⁸. Its free plan includes 10,000 monthly step executions without requiring credit card registration⁷.

Transactional communications deploy a dual-provider fallback architecture utilizing Resend as the primary transport and Brevo as the secondary failover⁹. Resend furnishes 3,000 emails per month free of charge⁹, while Brevo provides 300 daily emails (totaling up to 9,000 monthly) with support for unlimited stored contacts¹⁰. This dual-engine strategy protects against daily rate limits during unexpected spikes in patient communications⁹.

Generative AI processing for patient intake triage and localized post-visit summary formatting uses the Google GenAI SDK (@google/genai) to access the gemini-2.5-flash model¹⁴. This configuration supplies high-speed inference, strict structured JSON schema enforcement, and access under Google AI Studio's free tier quotas¹⁴.

Infrastructure Layer	Technology Provider	Free Tier Operational Limits	Core Technical Responsibility	Cold-Start & Resource Exhaustion Mitigation Strategy
----------------------	---------------------	------------------------------	-------------------------------	--

Relational Storage	Neon PostgreSQL ³	0.5 GB Storage, 100 CU-hrs/mo, Compute Auto-suspend ³	Persistent patient records, appointment states, EHR data ⁴	Stateless HTTP driver to bypass TCP connection penalties; optimistic UI state updates ⁴
Distributed Lock / Cache	Upstash Redis ⁵	256 MB Storage, 500,000 Commands/mo ⁵	Edge concurrency locks, rate-limiting, session state ⁵	REST-based stateless SDK; strict TTL key eviction policies to retain 256 MB boundary ⁵
Workflow Engine	Inngest ⁷	10,000 Step Executions/mo ⁷	Leave propagation sagas, async notifications, retry queues ⁷	Step execution batching; exponential backoff execution with persistent step caching ⁷
Transactional Email	Resend ⁹ & Brevo ¹⁰	Resend: 3k/mo ⁹ ; Brevo: 300/day ¹⁰	Reservation confirmations, reschedule alerts, follow-up links ⁹	Automatic provider failover; daily volume tracking in Upstash Redis ⁹
Generative Intelligence	@google/genai (Gemini) ¹⁴	Free Tier (Gemini 2.5 Flash / 15 RPM) ¹⁴	Symptom intake synthesis, localized discharge formatting ¹⁴	Schema-enforced JSON responses; localized prompt template caching ¹⁵
Application Layer	Next.js 14/15 App Router	Vercel Hobby Tier	Server-Side Rendering (SSR), Server	Static site generation (SSG) for static

			Actions, Middleware ¹⁷	pages; localized layout streaming
--	--	--	--------------------------------------	---

High-Concurrency Scheduling Engine & Race-Condition Guardrails

Preventing double-booking in medical appointment systems presents a significant challenge. When two users concurrently request the same doctor's schedule slot, naive check-then-insert database patterns introduce a time-of-check to time-of-use (TOCTOU) race condition. If two requests evaluate slot availability simultaneously, both may find the slot open and execute write operations, resulting in conflicting bookings.

To prevent race conditions across distributed serverless instances, the system implements a two-tier concurrency control strategy: an edge-level distributed mutex managed via Upstash Redis, combined with pessimistic row-level locking and partial unique index constraints in PostgreSQL.

When a patient initiates a slot reservation, the request lands on a Next.js Server Action or API Route Handler. The runtime immediately attempts to acquire an exclusive, short-lived distributed lock in Upstash Redis using atomic commands.

```
// Lock key construction targeting doctor ID and specific timestamp
const lockKey = `lock:doctor:${doctorId}:slot:${slotTimestamp.getTime()}`;
const lockToken = crypto.randomUUID();
const ttlMilliseconds = 5000; // 5-second lock window

// Execute atomic SET if Not Exists (NX) with Expiration (PX)
const lockAcquired = await redis.set(lockKey, lockToken, {
  nx: true,
  px: ttlMilliseconds,
});

if (!lockAcquired) {
  throw new SlotConflictException(
    "This appointment slot is currently being processed by another user. Please select another time."
  );
}
```

Once the Redis lock is secured, execution moves to the database layer within a isolated

PostgreSQL transaction. The engine issues a `SELECT ... FOR UPDATE NOWAIT` query against the target slot. This statement locks the matching database row immediately. If another process holds an active lock on that row, PostgreSQL rejects the secondary operation without waiting, preventing thread pool exhaustion.

To guarantee absolute database consistency even during cache eviction or key expiration, the underlying PostgreSQL database schema enforces a partial unique index. This constraint prevents multiple active bookings from sharing the same doctor and timestamp.

```
-- Enforce database-level uniqueness for active appointments
CREATE UNIQUE INDEX idx_prevent_double_booking
ON appointments (doctor_id, appointment_timestamp)
WHERE status IN ('PENDING', 'CONFIRMED');
```

```
import { db } from "@lib/db";
import { appointments, slots } from "@lib/db/schema";
import { eq, and, sql } from "drizzle-orm";

export async function processSlotBooking(
  string doctorId,
  string timestamp,
  Date appointmentTimestamp
) {
  Promise { success: boolean; appointmentId?: string; error?: string }> {
    try {
      return await db.transaction(async (tx) => {
        // 1. Acquire row lock with NOWAIT to prevent blocking
        const slotQuery = await tx.execute(
          sql`SELECT id, status FROM slots
            WHERE doctor_id = ${doctorId}
            AND slot_timestamp = ${slotTimestamp}
            FOR UPDATE NOWAIT`
        );

        if (!slotQuery.rows.length) {
          throw new Error("SLOT_NOT_FOUND");
        }
      });
    }
  }
}
```

```

    }

    const currentSlot = slotQuery.rows[0] as { id: string; status: string };

    if (currentSlot.status !== "AVAILABLE") {
        throw new Error("SLOT_UNAVAILABLE");
    }

    // 2. Transition slot status to RESERVED
    await tx
        .update(slots)
        .set({ status: "RESERVED" })
        .where(eq(slots.id, currentSlot.id));

    // 3. Insert appointment record
    const [newAppointment] = await tx
        .insert(appointments)
        .values({
            patientId,
            doctorId,
            appointmentTimestamp: slotTimestamp,
            status: "PENDING",
            createdAt: new Date(),
        })
        .returning({ id: appointments.id });

    return { success: true, appointmentId: newAppointment.id };
});
} catch (error: any) {
    // Catch database unique constraint violations (PostgreSQL error 23505)
    if (error.code === "23505" || error.message === "SLOT_UNAVAILABLE") {
        return {
            success: false,
            error: "CONFLICT_DOUBLE_BOOKING",
        };
    }
    throw error;
}
}

```

This dual-layer mechanism guarantees isolation. The Upstash Redis lock absorbs high-frequency concurrent traffic at the network edge⁵, while the PostgreSQL row locks and partial indexes ensure strict ACID compliance within the persistence layer.

Asynchronous Doctor Leave Propagation Pipeline

Medical practice management requires handling unscheduled physician absences. When a doctor requests emergency or extended leave, all conflicting appointments across the leave window must be invalidated, database resources released, priority reschedule tokens generated, and impacted patients notified. Attempting to execute these operations synchronously inside a single HTTP request risks runtime timeouts, memory breaches, and partial transaction failures.

The system addresses this by using an asynchronous saga pattern orchestrated through Inngest⁷. The initial API endpoint validates the leave request, records the leave block in the database, dispatches a doctor/leave.declared event to the Inngest engine, and returns an immediate HTTP 202 Accepted response to the client application.

Upon capturing the event, Inngest executes a multi-step background function equipped with automatic step retries and isolated state checkpoints⁷.

```
import { Inngest } from "inngest";
import { db } from "@lib/db";
import { appointments, slots } from "@lib/db/schema";
import { eq, and, gte, lte, inArray } from "drizzle-orm";
import { sendEmailWithFallback } from "@lib/email-provider";

export const inngest = new Inngest({ id: "healthcare-platform" });

export const propagateDoctorLeaveSaga = inngest.createFunction(
  {
    id: "propagate-doctor-leave-saga",
    retries: 5,
  },
  { event: "doctor/leave.declared" },
  async ({ event, step }) => {
    const { doctorId, startDate, endDate, reason } = event.data;

    // Step 1: Bulk update slots and invalidate active appointments
    const cancelledAppointments = await step.run("cancel-conflicting-bookings", async () => {
      return await db.transaction(async (tx) => {
        // Mark all slots in range as UNAVAILABLE
        await tx
          .update(slots)
          .set({ status: "UNAVAILABLE" })
```

```

.where(
  and(
    eq(slots.doctorId, doctorId),
    gte(slots.slotTimestamp, new Date(startDate)),
    lte(slots.slotTimestamp, new Date(endDate))
  )
);

// Cancel pending and confirmed appointments
return await tx
  .update(appointments)
  .set({
    status: "CANCELLED_BY_PROVIDER",
    cancellationReason: reason || "Provider Emergency Leave",
  })
  .where(
    and(
      eq(appointments.doctorId, doctorId),
      gte(appointments.appointmentTimestamp, new Date(startDate)),
      lte(appointments.appointmentTimestamp, new Date(endDate)),
      inArray(appointments.status, ["PENDING", "CONFIRMED"])
    )
  )
  .returning({
    id: appointments.id,
    patientId: appointments.patientId,
    patientEmail: appointments.patientEmail,
    patientLanguage: appointments.patientLanguage,
    appointmentTimestamp: appointments.appointmentTimestamp,
  });
});

if (cancelledAppointments.length === 0) {
  return { status: "COMPLETED", processedRecords: 0 };
}

// Step 2: Generate priority rescheduling tokens stored in Upstash Redis
const reschedulePayloads = await step.run("generate-reschedule-tokens", async () => {
  const payloads = [];
  for (const appt of cancelledAppointments) {
    const priorityToken = crypto.randomUUID();
    // Cache priority token in Redis with a 72-hour expiration window

```



```

    await redis.set(`reschedule:token:${priorityToken}`, JSON.stringify({
      patientId: appt.patientId,
      originalAppointmentId: appt.id,
      doctorId: doctorId,
    }), { px: 259200000 });

    payloads.push({
      ...appt,
      priorityToken,
    });
  }
  return payloads;
});

// Step 3: Batch process patient notifications using transactional failover
for (const payload of reschedulePayloads) {
  await step.run(`notify-patient-${payload.id}`, async () => {
    await sendEmailWithFallback({
      to: payload.patientEmail,
      subject: "Important Notice: Your Appointment Needs to Be Rescheduled",
      template: "doctor-leave-notification",
      locale: payload.patientLanguage || "en",
      context: {
        appointmentDate: payload.appointmentTimestamp,
        rescheduleLink:
          `https://platform.domain/reschedule?token=${payload.priorityToken}&locale=${payload.patientLanguage}`,
      },
    });
  });
}

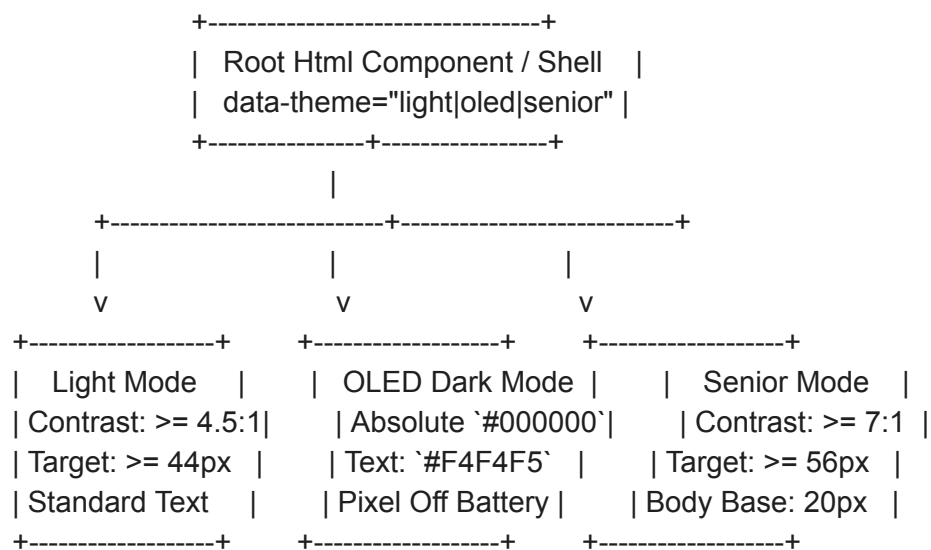
return { status: "SUCCESS", processedRecords: cancelledAppointments.length };
}
);

```

This workflow ensures reliability. If a transactional email provider experiences an outage mid-stream, Inngest resumes execution at the exact failed notification step without duplicating database operations or re-canceling appointments⁷.

Tri-Mode Accessibility & Adaptive Interface Architecture

Healthcare software serves users across diverse age groups, visual capabilities, and physical environments. To accommodate these needs, the platform provides a tri-mode accessibility system supporting Light Mode, OLED Dark Mode, and Senior Accessibility Mode.



Light Mode targets standard clinical and desktop environments, maintaining compliance with WCAG 2.1 AA standards (minimum contrast ratio of 4.5:1). OLED Dark Mode sets surface backgrounds to absolute pitch black (#000000). On mobile devices with OLED screens, pure black pixels turn off entirely, reducing display power consumption while providing comfortable viewing in low-light environments. Text elements use high-luminance off-white shades (#F4F4F5) to prevent optical blooming and maintain a 7:1 contrast ratio (WCAG AAA). Senior Accessibility Mode addresses age-related visual degradation, tremors, and reduced fine motor control. Base typography scales up by 125% (starting from a 20px body base), minimum interactive touch targets expand from 44px to 56px, visual borders increase to 2px solid contrast lines, and contrast ratios exceed 10:1. Visual indicators avoid relying solely on subtle color shifts, pairing every state change with explicit text labels and high-visibility icons.

Design System Dimension	Light Mode (Standard)	OLED Dark Mode (Power/Low-Light)	Senior Accessibility Mode
Primary Root Background	Slate 50 (#F8FAFC)	Pure Absolute Black (#000000)	Warm Soft Cream (#FAF9F6)
Surface Element	Pure White	Deep Zinc	High-Contrast Pure

Background	(#FFFFFF)	(#09090B)	White (#FFFFFF)
Primary Text Color	Slate 900 (#0F172A)	Zinc 100 (#F4F4F5)	Solid Pitch Black (#000000)
Border & Divider Accent	Slate 200 (#E2E8F0)	Zinc 800 (#27272A)	Heavy Pitch Black (#000000)
Target Minimum Contrast	4.5:1 (WCAG AA Compliant)	7:1 (WCAG AAA Compliant)	10:1+ (Ultra High Contrast)
Base Body Text Sizing	16px (1.00rem)	16px (1.00rem)	20px (1.25rem)
Minimum Touch Target	44px x 44px	44px x 44px	56px x 56px
Interactive Border Width	1px Solid	1px Solid	2px Solid High-Contrast
Typography Weight Scale	Regular (400) / Medium (500)	Regular (400) / Medium (500)	Semi-Bold (600) / Bold (700)

```
/* src/app/globals.css */
@layer base {
  :root[data-theme="light"] {
    --bg-root: 210 40% 98%;
    --surface-card: 0 0% 100%;
    --text-primary: 222.2 84% 4.9%;
    --text-muted: 215.4 16.3% 46.9%;
    --          : 214.3 31.8% 91.4%;
    --primary-action: 221.2 83.2% 53.3%;
    --    -scale: 1;
    --touch-target-min: 44px;
```

```

--          -main: 1px;
}

:root[data-theme="oled"] {
  --bg-root: 0 0% 0%; /* Pure #000000 turns off pixels on OLED displays */
  --surface-card: 240 10% 3.9%;
  --text-primary: 0 0% 98%;
  --text-muted: 240 5% 64.9%;
  --          : 240 3.7% 15.9%;
  --primary-action: 217.2 91.2% 59.8%;
  --    -scale: 1;
  --touch-target-min: 44px;
  --          -main: 1px;
}

:root[data-theme="senior"] {
  --bg-root: 40 20% 97%; /* Warm non-glare cream background */
  --surface-card: 0 0% 100%;
  --text-primary: 0 0% 0%; /* Max contrast black text */
  --text-muted: 0 0% 20%;
  --          : 0 0% 0%; /* High contrast solid black borders */
  --primary-action: 220 100% 20%; /* Deep navy blue */
  --    -scale: 1.25; /* Scaled base typography size */
  --touch-target-min: 56px; /* Expanded touch targets for motor impairment */
  --          -main: 2px;
}
}

.accessible-action-button {
  : var(--touch-target-min);
  : var(--touch-target-min);
  : calc(1rem * var(--font-scale));
  : var(--border-width-main);
}

```

Trilingual Localization & Indian Script Typographic Stack

Supporting English, Tamil (தமிழ்), and Hindi (हिन्दी) requires handling distinct internationalization (i18n) patterns and font configurations. The application uses next-intl within the Next.js App Router for server-side localization and locale-aware routing¹⁷.

```

src/
├── app/
│   ├── [locale]/
│   │   ├── layout.tsx    <-- Dynamic font class assignment & NextIntlClientProvider
│   │   ├── page.tsx     <-- Page component rendering localized strings
│   │   └── appointments/
│   └── i18n/
│       ├── request.ts    <-- Request-scoped locale resolution logic
│       └── messages/
│           ├── en.json    <-- English translation keys
│           ├── ta.json    <-- Tamil translation keys
│           └── hi.json    <-- Hindi translation keys

```

Devanagari (Hindi) and Tamil scripts feature intricate typographic characteristics, including complex ligatures, upper/lower diacritics (matras), and distinct vertical metric heights. Standard system fallbacks often fail to render these scripts correctly, resulting in missing character glyph boxes ("tofu") or overlapping line heights¹⁹.

To fix this, the application integrates `next/font/google` to dynamically load Noto Sans (English), Noto Sans Devanagari (Hindi), and Noto Sans Tamil (Tamil)¹⁹. Variable font loading ensures glyphs render properly without introducing layout shifts or downloading unnecessary font weights²¹.

```

// src/lib/fonts.ts
import { Noto_Sans, Noto_Sans_Devanagari, Noto_Sans_Tamil } from "next/font/google";

export const fontLatin = Noto_Sans({
  subsets: ["latin"],
  variable: "--font-latin",
  display: "swap",
  weight: ["400", "500", "600", "700"],
});

export const fontDevanagari = Noto_Sans_Devanagari({
  subsets: ["devanagari"],
  variable: "--font-hindi",
});

```

```

    display: "swap",
    weight: ["400", "500", "600", "700"],
  });

```

```

export const fontTamil = Noto_Sans_Tamil({
  subsets: ["tamil"],
  variable: "--font-tamil",
  display: "swap",
  weight: ["400", "500", "600", "700"],
});

```

```

// src/app/[locale]/layout.tsx
import "@app/globals.css";
import { fontLatin, fontDevanagari, fontTamil } from "@lib/fonts";
import { NextIntlClientProvider } from "next-intl";
import { getMessages } from "next-intl/server";
import { notFound } from "next/navigation";

const allowedLocales = ["en", "ta", "hi"];

export default async function RootLocaleLayout

```

```

    Promise      string
  {
    const { locale } = await params;

    if (!allowedLocales.includes(locale)) {
      notFound();
    }

    const messages = await getMessages();

    // Resolve active font variable based on route locale
    const resolveActiveFontClass = {
      switch (locale) {

```

```

    case "ta":
      return `${fontTamil.variable} font-tamil`;
    case "hi":
      return `${fontDevanagari.variable} font-hindi`;
    default:
      return `${fontLatin.variable} font-sans`;
  }
};

return (
  

```

```

// src/i18n/request.ts
import { getRequestConfig } from "next-intl/server";

export default getRequestConfig(async ({ requestLocale }) => {
  let locale = await requestLocale;

  if (!locale || ![ "en", "ta", "hi" ].includes(locale)) {
    locale = "en";
  }

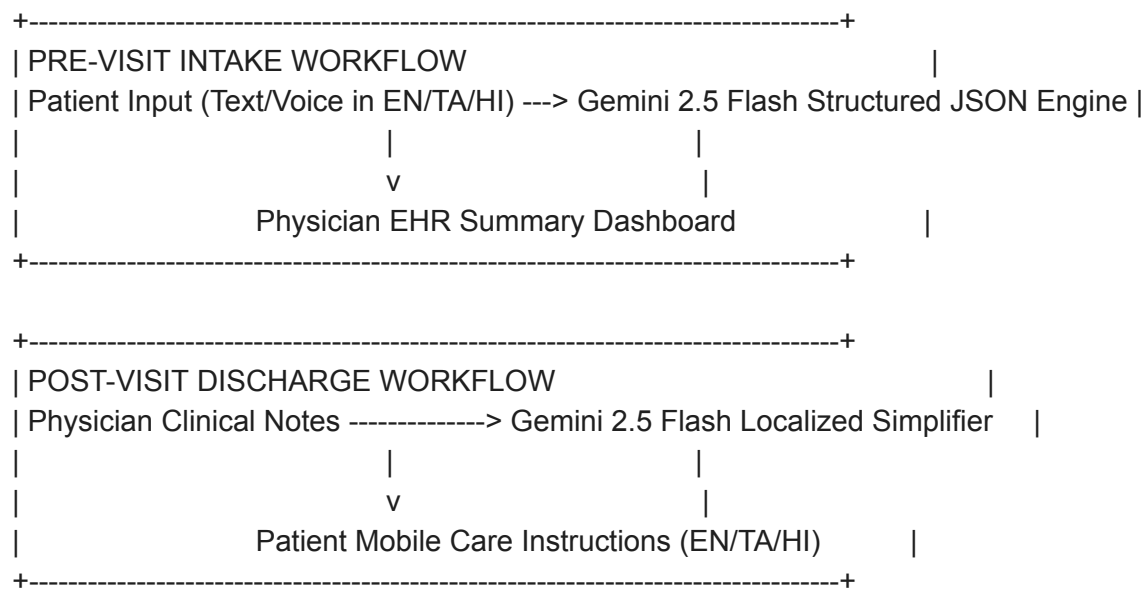
  return {
    locale,
    messages: (await import(`../../messages/${locale}.json`)).default,
  };
});

```

LLM-Driven Pre-Visit Triage & Post-Visit Summary

Workflows

Integrating generative AI into clinical operations improves patient data collection before consultations and simplifies post-visit care instructions afterward. The system uses the Google GenAI SDK (@google/genai) to run structured inference through gemini-2.5-flash¹⁴.



Pre-Visit Intake Workflow

Before a consultation, patients submit their primary complaints, symptom duration, and medical history in English, Tamil, or Hindi. The pre-visit triage engine parses this input and returns a structured JSON payload for the physician's intake dashboard¹⁶.

```
// src/lib/ai/intake-agent.ts
import { GoogleGenAI, Type, Schema } from "@google/genai";

const ai = new GoogleGenAI({ apiKey: process.env.GEMINI_API_KEY! });

const preVisitIntakeSchema: Schema = {
  type: Type.OBJECT,
  properties: {
```



```

chiefComplaint: { type: Type.STRING },
symptomDurationDays: { type: Type.INTEGER },
painScaleOneToTen: { type: Type.INTEGER },
redFlagAlerts: {
  type: Type.ARRAY,
  items: { type: Type.STRING },
},
formattedClinicalSummary: { type: Type.STRING },
},
required: [
  "chiefComplaint",
  "symptomDurationDays",
  "painScaleOneToTen",
  "redFlagAlerts",
  "formattedClinicalSummary",
],
};

```

```

export async function processPatientIntake

```

```

    string
    "en" "ta" "hi"

```

```

{
  const systemInstruction = `
    You are an expert emergency medicine triage assistant.
    Analyze the incoming patient symptom description provided in English, Tamil, or Hindi.
    Extract key clinical metrics and generate a concise EHR pre-visit summary in English for the
    doctor.
    Identify any emergency red-flag symptoms (e.g., chest pain, acute dyspnea, sudden neurological
    deficits).
  `;

```

```

const response = await ai.models.generateContent({
  model: "gemini-2.5-flash",
  contents: `Patient Submission (${inputLanguage}): ${rawInputText}`,
  config: {
    systemInstruction,
    responseMimeType: "application/json",
    responseSchema: preVisitIntakeSchema,
    temperature: 0.1, // Low temperature for deterministic output
  },
});

```

```

return JSON.parse(response.text!);
}

```

Post-Visit Discharge Simplification Workflow

After a consultation, the attending physician enters clinical notes and treatment plans. The post-visit simplification engine converts these notes into plain-language discharge instructions translated into the patient's preferred language (English, Tamil, or Hindi).

```
// src/lib/ai/discharge-agent.ts
import { GoogleGenAI, Type, Schema } from "@google/genai";

const ai = new GoogleGenAI({ apiKey: process.env.GEMINI_API_KEY! });

const postVisitDischargeSchema: Schema = {
  type: Type.OBJECT,
  properties: {
    simplifiedDiagnosis: { type: Type.STRING },
    medicationInstructions: {
      type: Type.ARRAY,
      items: {
        type: Type.OBJECT,
        properties: {
          medicationName: { type: Type.STRING },
          purpose: { type: Type.STRING },
          timingAndDosage: { type: Type.STRING },
        },
      },
    },
    homeCareSteps: {
      type: Type.ARRAY,
      items: { type: Type.STRING },
    },
    warningSignsToReturn: {
      type: Type.ARRAY,
      items: { type: Type.STRING },
    },
  },
  required: [
    "simplifiedDiagnosis",
    "medicationInstructions",
  ],
}
```

```

    "homeCareSteps",
    "warningSignsToReturn",
  ],
};

export async function generateLocalizedDischargeSummary
    string
    "en" "ta" "hi"

{
  const languageMap = {
    en: "English",
    ta: "Tamil",
    hi: "Hindi",
  };

  const systemInstruction = `
    You are a patient advocate and medical communicator.
    Translate the physician's clinical notes into clear, 4th-grade reading level discharge instructions.
    Produce the output in ${languageMap[targetLanguage]}.
    Ensure all medication schedules, care steps, and emergency warning signs are easy to read and
    unambiguous.
  `;

  const response = await ai.models.generateContent({
    model: "gemini-2.5-flash",
    contents: `Physician Clinical Notes: ${physicianNotes}`,
    config: {
      systemInstruction,
      responseMimeType: "application/json",
      responseSchema: postVisitDischargeSchema,
      temperature: 0.2,
    },
  });

  return JSON.parse(response.text!);
}

```

Production Execution Prompt for Google Antigravity

The following production prompt contains full structural and implementation details to instantiate the platform within Google Antigravity (antigravity-preview using Gemini 3.6 Flash / Managed Agents)¹⁵.

SYSTEM SETUP PROMPT: FULL-STACK HEALTHCARE PLATFORM ENGINE

ROLE & CONTEXT

You are an Enterprise Software Architect and Principal Full-Stack Engineer. You are tasked with generating a production-ready, fully implemented codebase for an enterprise Healthcare Appointment & Follow-Up Platform operating under a zero-cost infrastructure architecture.

ARCHITECTURAL & TECH STACK SPECIFICATIONS

1. FOUNDATIONAL STACK

- Framework: Next.js 15 (App Router, React 19, Server Actions, Edge Middleware)
- Database: Serverless Neon PostgreSQL using @neondatabase/serverless and Drizzle ORM
- Distributed Cache & Locking: Upstash Redis (@upstash/redis) over stateless HTTP REST
- Background Workflows: Inngest (inngest) for asynchronous event processing
- Transactional Messaging: Resend (resend) with automatic failover to Brevo (@getbrevo/brevo)
- Generative AI: Google GenAI SDK (@google/genai) using gemini-2.5-flash
- Internationalization: next-intl supporting English (en), Tamil (ta), and Hindi (hi)
- Typography: next/font/google importing Noto_Sans, Noto_Sans_Devanagari, and Noto_Sans_Tamil
- Styling & Accessibility: Tailwind CSS with CSS custom properties supporting light, oled, and senior themes

2. CORE MODULE SPECIFICATIONS TO GENERATE

MODULE A: High-Concurrency Booking Engine

- File: src/lib/services/booking-service.ts
- Implement two-tier concurrency control:
 1. Acquire Redis lock via @upstash/redis: SET lock:doc:{id}:slot:{timestamp} token NX PX 5000
 2. Open PostgreSQL transaction using Drizzle ORM and execute SELECT ... FOR UPDATE NOWAIT
 3. Handle unique constraint violations (DB error 23505) and return localized HTTP 409 Conflict responses

MODULE B: Asynchronous Doctor Leave Propagation Pipeline

- File: src/inngest/functions/doctor-leave.ts
- Implement an event saga listening for doctor/leave.declared:
 1. Step 1: Bulk update slots status to UNAVAILABLE and affected appointments to CANCELLED_BY_PROVIDER
 2. Step 2: Generate priority rescheduling tokens stored in Upstash Redis (72h TTL)
 3. Step 3: Trigger patient email notifications using Resend with failover to Brevo

MODULE C: Tri-Mode Accessibility System

- Files: src/components/theme-provider.tsx & src/app/globals.css
- Support three distinct theme modes via data-theme on the <html> root element:
 - light: Standard clinical UI (contrast ratio $\geq 4.5:1$)
 - oled: True black background (#000000, contrast ratio $\geq 7:1$)
 - senior: High contrast ($\geq 10:1$), base font size 20px (1.25rem), touch targets ≥ 56 px

MODULE D: Trilingual i18n & Font Engine

- Files: src/app/[locale]/layout.tsx & src/i18n/request.ts
- Implement next-intl localized routing for /en, /ta, and /hi
- Bind Google Fonts dynamically based on locale:
 - English: Noto Sans
 - Hindi: Noto Sans Devanagari
 - Tamil: Noto Sans Tamil

MODULE E: LLM Pre & Post-Visit Workflows

- Files: src/lib/ai/intake-agent.ts & src/lib/ai/discharge-agent.ts
- Implement Google GenAI SDK integration (gemini-2.5-flash):
 1. processPatientIntake: Parse raw symptom submissions in EN/TA/HI and return structured JSON intake data (Chief Complaint, Red Flags, EHR Summary)
 2. generateLocalizedDischargeSummary: Translate clinical notes into simplified, 4th-grade level discharge guides in the patient's preferred language

3. DATABASE SCHEMA REQUIREMENTS

Define the schema in src/db/schema.ts using Drizzle ORM:

- doctors: id (uuid), name (text), specialty (text), languagesSpoken (jsonb)
- slots: id (uuid), doctorId (uuid), slotTimestamp (timestamp), status (enum: AVAILABLE, RESERVED, UNAVAILABLE)
- appointments: id (uuid), patientId (uuid), doctorId (uuid), appointmentTimestamp (timestamp), status (enum: PENDING, CONFIRMED, CANCELLED_BY_PATIENT, CANCELLED_BY_PROVIDER), cancellationReason (text)
- Index: Enforce a partial unique index idx_prevent_double_booking on appointments(doctorId, appointmentTimestamp) where status IN ('PENDING', 'CONFIRMED')

4. CODE GENERATION REQUIREMENTS

Generate full source code for all specified files. Ensure all imports, TypeScript interfaces, environment variable validations, and error handlers are fully written out. Do not use placeholder code or incomplete implementations. Begin generation immediately.

1.